# PROVISIONAL PATENT APPLICATION

## SYSTEM AND METHOD FOR COMPUTING ON PRIVATE DATA WITHOUT EXPOSURE THROUGH SANDBOXED TRANSFORM EXECUTION AND ENCRYPTED RELAY ARCHITECTURE

Authors: Tom Tang

Organization: Edge Vector Foundation

Date: March 30, 2026

---

## Applicant

Edge Vector Foundation
2328 Delmar Dr E, Seattle, WA 98102

## Inventors

- Tom Tang, Seattle, WA

## Field of the Invention

This invention relates to privacy-preserving computation on user-controlled data, and more particularly to a system and method for enabling third-party applications to derive computed results from private data without the data leaving the user's device or being exposed to any intermediary.

## Cross-Reference to Related Applications

None (provisional filing).

## Background of the Invention

Modern applications require access to user data — health records, financial information, personal documents, proprietary code — to provide useful functionality. Current architectures require users to upload data to centralized servers, where applications operate on it directly. This creates fundamental privacy problems: sensitive data is transmitted, stored, and processed on infrastructure the

user does not control. Once data reaches a platform, users have no meaningful control over how it is used, who sees it, or whether it is truly deleted.

Existing approaches to privacy-preserving computation, such as fully homomorphic encryption (FHE) and secure multi-party computation (SMPC), suffer from computational overhead that makes them impractical for general-purpose data operations. FHE imposes 1000x–10000x performance penalties. SMPC requires coordination between multiple parties and specialized protocols for each computation.

There is a need for a practical system that enables third-party applications to compute on private data without the data leaving the user's control, using commodity hardware and standard programming models.

## Summary of the Invention

The present invention provides a system and method for privacy-preserving computation comprising:

1. **A user-controlled database node** that stores data locally, encrypted at rest with keys derived from a user passkey, and serves as the sole authority over data access and computation.
2. **A schema-enforced access control layer ("folds")** that defines per-field permissions, trust distance requirements, security labels, and payment gates, evaluated at query time on the user's node.
3. **A sandboxed WebAssembly (WASM) transform execution engine** that enables third-party computation on private data within the user's node, where:
   - Transforms are content-addressed WASM modules registered in a public registry
   - Transforms execute in a sandboxed environment with no filesystem, network, or system access
   - Transforms operate on typed field values with declared input/output types verified at registration time
   - Transforms support three-layer composition: public standard library transforms, composed transforms chaining multiple operations, and encrypted wrapper transforms with embedded cryptographic keys
   - The node executes transforms locally — plaintext data never leaves the node
4. **An encrypted relay layer** that routes end-to-end encrypted requests between third-party applications and user nodes, where the relay validates authentication tokens and enforces mechanical limits (access counts, expiration) but never sees plaintext data, never makes access control decisions, and never executes transforms.
5. **A three-tier security model** comprising:
   - Tier 1: Plaintext transforms on public data with full verifiability

- Tier 2: WASM-encrypted transforms where cryptographic keys are embedded in WASM bytecode, with the sandbox preventing key exfiltration via I/O channels and bytecode obfuscation providing computational resistance to static key extraction
- Tier 3: WASM-encrypted transforms executed within a Trusted Execution Environment (TEE) providing hardware-attested memory isolation, eliminating the bytecode analysis attack vector through hardware-encrypted enclave memory

6. **A reversible transform system** where:
   - Transforms optionally export an inverse function
   - Invertibility is verified at registration time via round-trip testing with type-appropriate test inputs
   - Verified-reversible transforms enable write-back through the transform to the source data
   - Non-reversible transforms store write values directly, marking the source link as stale

## Detailed Description of the Invention

### System Architecture

The system comprises three principal components: (1) a user-controlled database node running locally on the user's device, (2) a public transform registry that stores content-addressed WASM modules, and (3) an encrypted relay service that routes communications without accessing plaintext.

#### User Node

The user node is a local database that:

- Stores all user data encrypted at rest using AES-256-GCM with keys derived from a user passkey via Argon2id key derivation and HKDF per-purpose key derivation
- Maintains an Ed25519 keypair for node identity and end-to-end encryption with applications
- Evaluates all access control policies locally at query time
- Executes all WASM transforms locally in a sandboxed runtime
- Returns only authorized, optionally transformed results to requesting applications

The node is the sole point of decryption. No external system — including the relay — ever sees plaintext data.

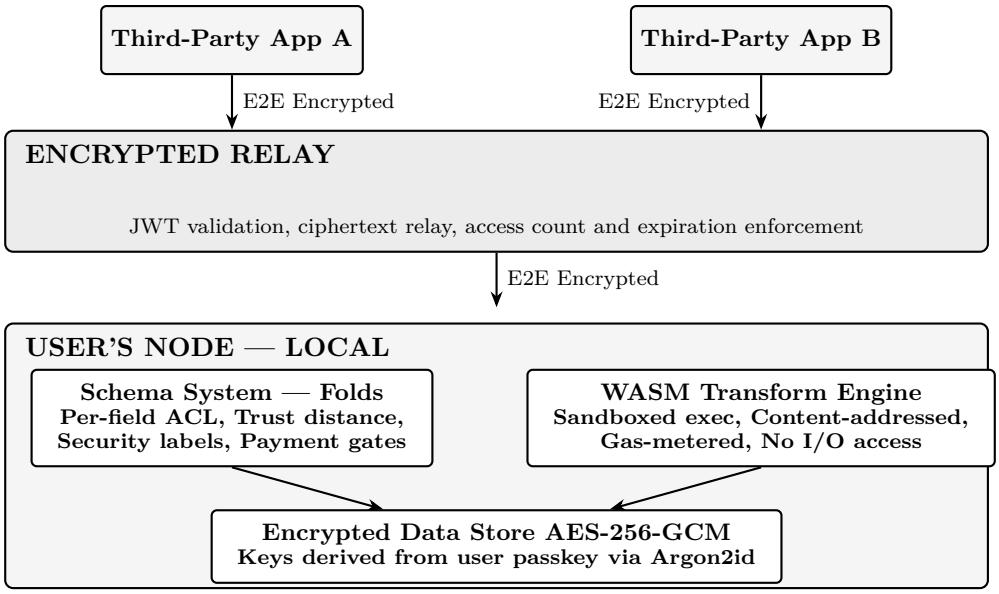



Figure 1: **System Architecture.** Third-party applications communicate with user nodes through an encrypted relay that validates tokens and relays ciphertext. All data decryption, access control, and transform execution occur on the user's local node.

### Schema System (Folds)

Data is accessed through named, versioned interface definitions called "folds." Each fold specifies:

- Field names and types
- Per-field access control: read/write permissions, trust distance requirements, security label classifications, and optional payment gates
- Optional transform functions that compute derived values from source data

Applications do not access raw data. All access goes through folds that enforce field-level policies. The query engine evaluates access control before returning data; unauthorized fields are excluded from results such that the requester cannot distinguish "field has no data" from "field exists but access is denied."

### WASM Transform Engine

Transforms are WebAssembly modules that compute derived values from source data. The transform engine uses a WASM runtime with:

- Ahead-of-time compilation for near-native execution speed
- Fuel metering (gas limits) to prevent infinite loops
- No system interface — no filesystem, network, clock, or any system access

- Deterministic execution mode with NaN canonicalization
- Per-module memory limits

Data passing between host and guest occurs through serialized field values copied into WASM linear memory. The WASM module cannot access host memory or any data outside its sandbox.

### Three-Layer Composition Model

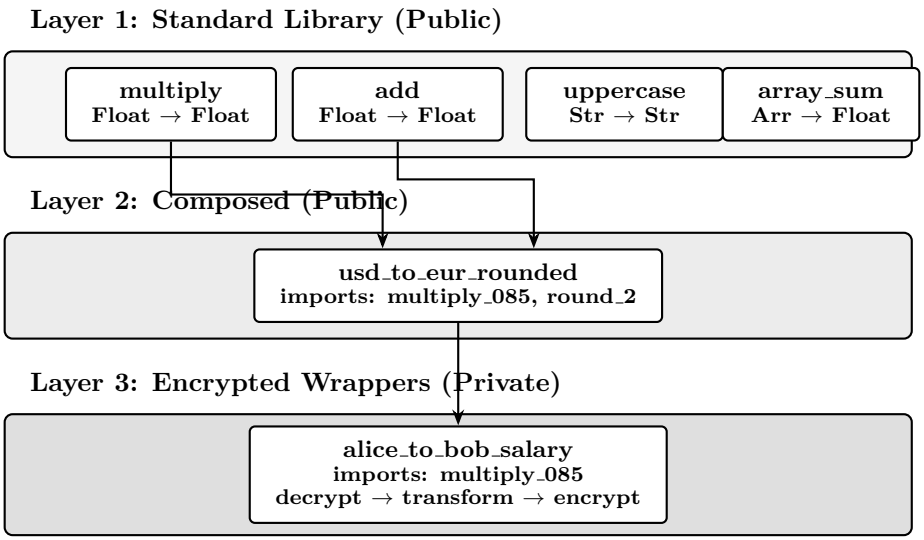



Figure 2: **Three-Layer Composition Model.** Public standard library transforms compose into chained transforms, which can be wrapped in encrypted wrappers with embedded cryptographic keys.

**Layer 1 — Standard Library (Public):** Pre-compiled WASM modules for common operations (arithmetic, string operations, array aggregation, classification). Published with source code for full public verifiability.

**Layer 2 — Composed Transforms (Public):** WASM modules that import and chain Layer 1 transforms. Composition is declared via WASM import tables, enabling verification of the computation chain without inspecting bytecode.

**Layer 3 — Encrypted Wrappers (Private):** User-generated WASM modules that wrap Layer 1 or 2 transforms with encryption/decryption logic. Cryptographic keys are embedded in the compiled WASM bytecode. The WASM sandbox prevents key exfiltration via I/O channels — no filesystem, network, or system call interfaces exist for keys to escape the sandbox boundary. The host runtime executes the module but cannot observe values within the WASM linear memory during execution.

*Note:* WASM bytecode obfuscation provides computational resistance to key extraction from the binary itself. For environments requiring hardware-guaranteed isolation, Layer 3 transforms may be executed within a Trusted Execution Environment (TEE), where the hardware encrypts enclave memory and provides remote attestation that the correct WASM module was executed faithfully. TEE execution eliminates the bytecode analysis attack vector by ensuring the host cannot inspect WASM memory even with physical access to the hardware.

An encrypted wrapper transform operates as follows:

1. Receives encrypted input (ciphertext bytes)
2. Decrypts using the data owner's key (embedded in WASM)
3. Applies the inner transform (a known, publicly verifiable Layer 1 or 2 transform)
4. Encrypts the result using the recipient's key (embedded in WASM)
5. Returns encrypted output (ciphertext bytes)

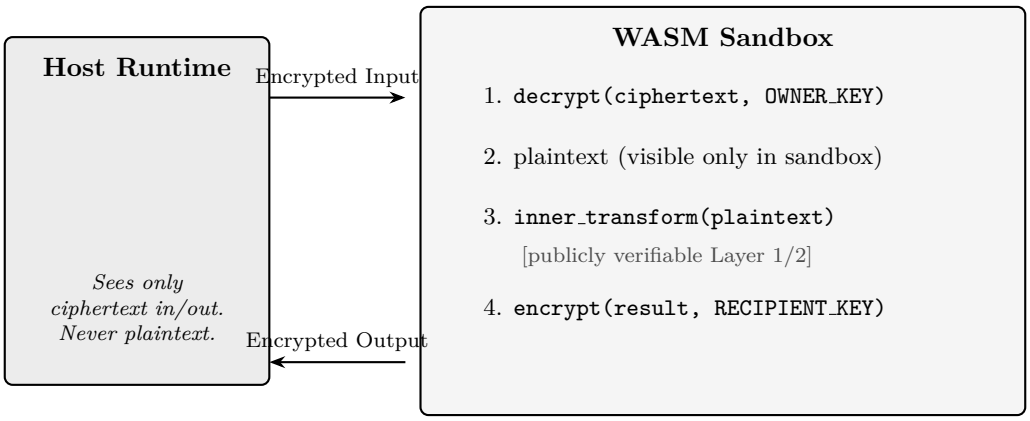



Figure 3: **Encrypted Wrapper Execution.** The host runtime passes ciphertext into the WASM sandbox, which decrypts, transforms, and re-encrypts. Plaintext and keys are never visible to the host.

The host sees bytes in, bytes out. The plaintext is never exposed outside the WASM sandbox.

### Transform Verification

At registration time, the system performs:

1. **Type compatibility checking** — transform input type must match source field type; output type must match target field type
2. **Encryption consistency** — encrypted field types require encrypted transform input/output types
3. **Import resolution** — all imported transforms must exist in the registry

4. **Acyclicity verification** — the import graph must be a directed acyclic graph
5. **Gas limit validation** — max gas within system-configured bounds
6. **Reversibility verification** — if declared reversible, round-trip testing with type-appropriate synthetic inputs confirms invertibility

### Content-Addressed Transform Registry

Each transform is identified by `SHA-256(wasm_bytes)`. This content-addressing provides:

- **Immutability:** once registered, a transform cannot be modified without changing its identity
- **Tamper evidence:** the node verifies `SHA-256(fetched_bytes) == expected_hash` before execution
- **Deduplication:** identical transforms produce identical hashes regardless of who registers them
- **Public auditability:** anyone can compile the published source and verify the hash matches

### Encrypted Relay

The relay service (Exemem) performs strictly mechanical functions:

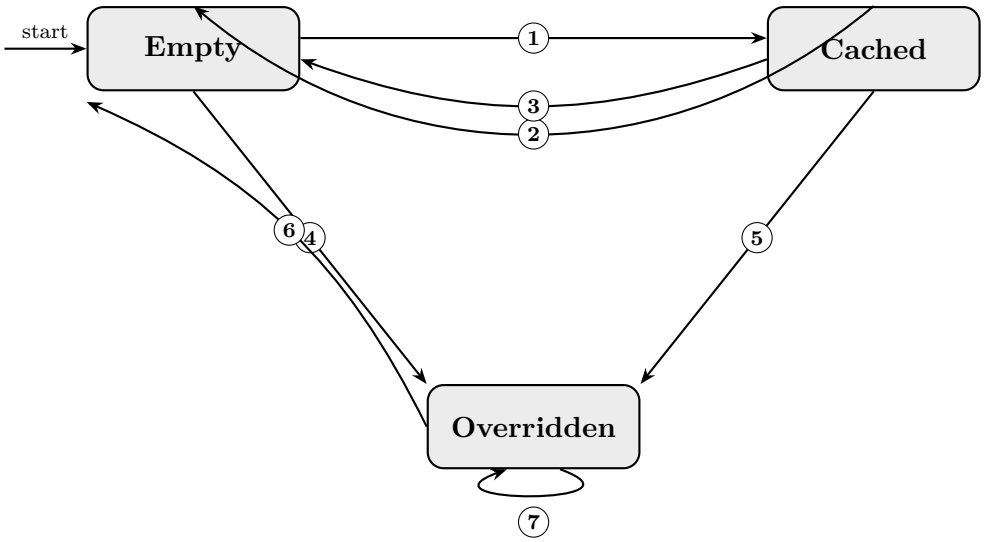
- Validates JWT authentication tokens (authenticity, expiration)
- Relays end-to-end encrypted ciphertext between applications and user nodes
- Enforces mechanical limits (access counts, time-based expiration)
- Stores encrypted blobs (node configuration, data backups)

The relay never decrypts data, never reads request or response bodies, never makes access control decisions, and never executes transforms. It is architecturally incapable of accessing plaintext.

### Transform Field State Machine

Transform fields operate under a three-state machine:

- **Empty**: Never accessed. No value computed or stored.
- **Cached**: Value computed from source via WASM transform. Recomputes when source changes (invalidation via dependency map).
- **Overridden**: Value directly written by user. Source link is stale. Persists until explicitly cleared.




① **Read on Empty:** compute via WASM → Cached
② **Reversible write:** run inverse WASM, mutate source → Empty
③ **Source mutation:** invalidate cache → Empty
④ **Irreversible write** (from Empty): store directly → Overridden
⑤ **Irreversible write** (from Cached): store directly → Overridden
⑥ **Reversible write** (from Overridden): inverse WASM → Empty
⑦ **Source mutation** on Overridden: no change (stays Overridden)

Figure 4: **Transform Field State Machine.** Fields transition between Empty, Cached, and Overridden states based on read, write, and source mutation events.

Transitions:

- **Read on Empty:** compute via WASM, transition to Cached
- **Read on Cached:** return cached value
- **Read on Overridden:** return stored value
- **Reversible write:** run inverse WASM, mutate source, field resets to Empty
- **Irreversible write:** store directly, transition to Overridden
- **Source mutation:** Cached resets to Empty; Overridden unchanged

## Data Flow — End-to-End Encrypted Computation

### Write Path (client encrypts):

1. Client encrypts plaintext field value with field-specific key
2. Server stores encrypted field value — knows only that it is "an encrypted [type]"

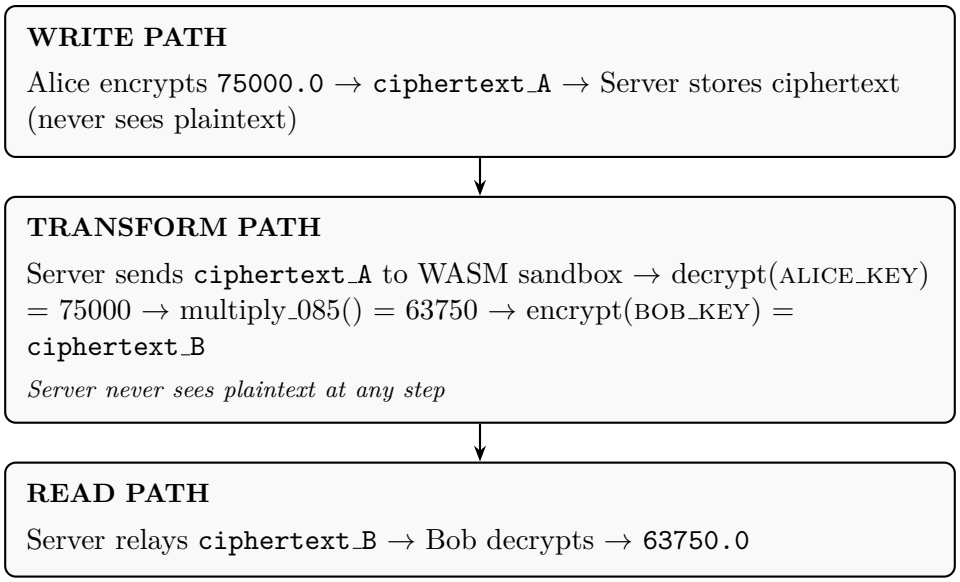



Figure 5: **End-to-End Encrypted Computation.** Alice encrypts and stores data. The WASM sandbox decrypts, transforms, and re-encrypts for Bob. The server never sees plaintext at any stage.

**Transform Path (WASM computes on ciphertext):**

1. Server loads registered WASM module by content hash
2. Server calls transform with encrypted input bytes
3. Inside WASM sandbox: decrypt with owner's key, apply inner transform, encrypt with recipient's key
4. Server receives encrypted output bytes — never sees plaintext

**Read Path (client decrypts):**

1. Server returns encrypted transform output
2. Recipient client decrypts locally with their key

## Claims

1. A computer-implemented method for computing on private data without exposure, comprising:
    a. storing data on a user-controlled node, encrypted at rest with keys derived from a user passkey;
    b. defining schema-enforced access control interfaces (folds) that specify per-field permissions, trust distance, security labels, and payment gates;

c. executing third-party computation as sandboxed WebAssembly modules on the user's node, wherein the modules have no access to filesystem, network, or system resources;

d. routing communications between applications and the user's node through an encrypted relay that validates authentication tokens and relays ciphertext without decrypting or interpreting data content;

e. evaluating all access control policies on the user's node at query time such that the relay and requesting applications cannot distinguish between nonexistent data and data for which access is denied.

2. The method of claim 1, wherein the WebAssembly transforms are content-addressed by the SHA-256 hash of their bytecode, registered in a public registry, and verified by the user's node before execution by recomputing the hash and comparing to the expected value.

3. The method of claim 1, wherein transforms support a three-layer composition model comprising:

   a. a public standard library of auditable transforms with published source code;

   b. composed transforms that chain standard library operations via declared WASM imports;

   c. encrypted wrapper transforms with cryptographic keys embedded in compiled WASM bytecode, where the WASM sandbox prevents key exfiltration via I/O channels and optionally executes within a Trusted Execution Environment for hardware-guaranteed memory isolation.

4. The method of claim 3, wherein an encrypted wrapper transform:

   a. receives ciphertext input from the host;

   b. decrypts the input using the data owner's key embedded in the WASM module;

   c. applies an inner transform that is a publicly verifiable Layer 1 or Layer 2 transform;

   d. encrypts the result using the recipient's key embedded in the WASM module;

   e. returns ciphertext output to the host;

   such that the host runtime never observes plaintext data.

5. The method of claim 1, further comprising a transform field state machine with three states — Empty, Cached, and Overridden — wherein:

   a. Empty fields are lazy-computed on first read via WASM transform execution;

   b. Cached fields are invalidated when source data changes via a dependency map;

   c. Overridden fields persist user-written values independently of source data changes;

   d. Reversible transforms verified at registration time via round-trip testing enable write-back through the inverse transform to mutate source data.

6. The method of claim 1, wherein the encrypted relay:

   a. stores only ciphertext and encrypted blobs;

   b. validates JWT tokens for authentication and quota enforcement;

c. is architecturally incapable of decrypting data, reading request/response bodies, or making access control decisions;

d. provides no degraded access mode when the user's node is offline — requests fail rather than falling back to cached or pre-decrypted data.

7. A system for privacy-preserving data computation comprising:

   a. a local database node configured to store data encrypted at rest and serve as the sole authority for decryption and access control;

   b. a schema system defining typed field-level access control interfaces;

   c. a sandboxed WASM runtime configured to execute content-addressed transform modules with no system access;

   d. a content-addressed transform registry storing immutable, publicly auditable WASM modules;

   e. an encrypted relay configured to route end-to-end encrypted communications between applications and user nodes without decrypting data.

8. The system of claim 7, wherein each fold field carries conjunctive access control comprising permissions, trust distance, security labels, and payment gates, all evaluated on the user's node at query time, with unauthorized fields excluded from results in a manner indistinguishable from nonexistent fields.

## Abstract

A system and method for enabling third-party applications to compute on private user data without the data leaving the user's device or being exposed to any intermediary. The system comprises a user-controlled database node that stores data encrypted at rest and evaluates all access control policies locally; a sandboxed WebAssembly transform execution engine that runs third-party computation within the node using content-addressed, publicly auditable WASM modules with declared type signatures and optional embedded encryption keys; and an encrypted relay that routes end-to-end encrypted communications between applications and user nodes without decrypting or interpreting data. Transforms support three-layer composition from public standard libraries through encrypted wrappers, enabling verifiable computation chains where the host runtime never observes plaintext. The system enables practical privacy-preserving computation on commodity hardware without the performance penalties of homomorphic encryption or multi-party computation.

*Prepared for provisional patent filing by Edge Vector Foundation.*
*Filing date: [TO BE DETERMINED]*
*This document constitutes a provisional patent application under 35 U.S.C. 111(b).*